

Attention Is All You Need (Transformer)— 논문 정리

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin (Google Brain / Google Research / University of Toronto)
NeurIPS 2017 · arXiv:1706.03762

1. 이 논문의 중요성

번역이나 언어 모델링 같은 시퀀스 변환(sequence transduction) 문제는 오랫동안 RNN(특히 LSTM, GRU) 기반 인코더-디코더 구조가 지배해왔다. 이 논문은 recurrence를 완전히 걷어내고 attention만으로 이를 대체할 수 있음을 보여줬는데, 단순히 "성능이 더 좋다"는 것을 넘어 학습 속도와 병렬화 측면에서 근본적인 이점을 제공한다는 점이 핵심이다.

2. 문제 정의: RNN의 구조적 한계

2-1. 순차 계산이라는 병목

RNN은 계산 단계를 시퀀스의 위치와 하나씩 짝지어 진행한다. 위치 t 의 은닉 상태 $h(t)$ 는 반드시 이전 단계의 은닉 상태 $h(t-1)$ 에 의존하기 때문에, t 번째 위치를 처리하려면 반드시 그 이전 위치들을 순서대로 먼저 처리해야 한다. 이 순차적 성질 때문에 하나의 학습 예시 안에서 병렬 계산이 근본적으로 불가능하고, 시퀀스가 길어질수록 메모리 제약으로 인해 여러 예시를 배치로 묶어 처리하기도 어려워진다. Factorization 기법이나 조건부 계산(conditional computation) 같은 방법들이 계산 효율을 상당히 개선했지만, 순차 계산이라는 근본적 제약 자체는 해소되지 않았다.

2-2. Attention의 부상

한편 attention 메커니즘은 입력이나 출력 시퀀스 안에서 두 요소 사이의 거리와 무관하게 의존 관계를 모델링할 수 있게 해주는 유용한 도구로, 이미 다양한 과제에서 시퀀스 모델의 핵심 구성 요소로 자리 잡고 있었다. 다만 극히 일부를 제외하면 대부분 RNN과 결합되어 보조적으로 쓰이고 있었다.

저자들은 "RNN을 아예 걷어내고 attention만으로 모델을 구성해 보자"는 제안을 한다. 이렇게 만든 Transformer는 훨씬 높은 수준의 병렬화를 가능하게 하고, 8개의 P100 GPU로 단 12시간만 학습해도 번역 품질에서 새로운 최고 수준에 도달할 수 있음을 보여준다.

3. Background — 관련 연구와의 차별점

계열	핵심 내용	Transformer와의 관계
Extended Neural GPU, ByteNet, ConvS2S	컨볼루션을 기본 구성 요소로 써서 입출력 위치의 은닉 표현을 병렬로 계산	"순차 계산 줄이기"라는 목표는 공유하지만, 두 위치 사이 신호를 연결하는 연산 횟수가 거리에 따라 늘어남(ConvS2S는 선형적으로, ByteNet은 로그적으로) → 장거리 의존성 학습이 어려움. Transformer는 이 연산 횟수를 상수로 줄이는 대신, attention 평균화로 인한 해상도 저하를 Multi-Head Attention으로 상쇄
Self-attention(intra-attention)	하나의 시퀀스 안에서 위치들을 연결해 표현을 계산하는 메커니즘. 독해, 요약, 텍스트 합성 등에서 이미 활용되던 개념	Transformer의 핵심 구성 요소로 채택되지만, 이 논문 이전까지는 대부분 RNN과 함께 쓰였음
End-to-end memory network	시퀀스 정렬 recurrence 대신, attention을 여러 번(hop) 반복 적용하는 recurrent attention 메커니즘 기반	시퀀스 위치가 아니라 "reasoning 단계"를 기준으로 반복한다는 점에서 Transformer의 self-attention과는 다른 접근

4. 모델 아키텍처

4-1. 전체 구조

인코더는 입력 시퀀스 $(x_1, \dots, x_n)$ 을 연속 표현 시퀀스 $z = (z_1, \dots, z_n)$ 으로 매핑하고, 디코더는 z 를 받아 출력 시퀀스 $(y_1, \dots, y_m)$ 을 한 번에 하나씩 자기회귀적(auto-regressive)으로 생성한다 — 이전에 생성한 심볼들을 추가 입력으로 다시 참고하면서 다음 심볼을 만든다.

4-2. 인코더/디코더 스택

인코더는 동일한 층 $N=6$ 개로 구성된다. 각 층은 두 서브레이어(multi-head self-attention, position-wise feed-forward network)를 가지며, 두 서브레이어 모두 residual connection을 두르고 그 뒤에 layer normalization을 적용한다 — 각 서브레이어의 출력은 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 형태이다. 잔차 연결이 성립하려면 입출력 차원이 같아야 하므로, 모든 서브레이어와 임베딩 레이어는 $d_{\text{model}} = 512$ 라는 동일한 차원을 출력한다.

디코더도 동일한 층 $N=6$ 개로 구성되지만, 두 가지 차이가 있다. 첫째, 인코더 스택의 출력에 대해 multi-head attention을 수행하는 서브레이어(encoder-decoder attention)가 하나 더 삽입된다. 둘째, self-attention 서브레이어가 뒤쪽 위치를 미리 내다보지 못하도록

masking된다. 이 마스킹은 출력 임베딩이 한 칸 밀려있다는 사실과 결합되어, 위치 i 의 예측이 오직 i 보다 앞선 위치의 이미 알려진 출력에만 의존하도록 보장한다.

4-3. Attention 메커니즘

Query, Key, Value: attention은 하나의 쿼리(질문에 해당)와 여러 키-값 쌍(각 위치의 "이름표"와 "내용물")을 입력받아, 쿼리와 각 키의 호환성을 점수화하고 그 점수 비율대로 값들을 가중합하는 연산이다.

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V \quad \dots \text{식 (1)}$$

쿼리와 키의 내적을 구해 관련성 점수를 계산하고, softmax로 비율(가중치)로 바꾼 뒤 값(V)을 가중합한다. d_k 가 커지면 내적값이 커져 softmax가 기울기가 거의 0인 영역으로 밀려나기 쉬운데, 이를 상쇄하기 위해 결과를 $\sqrt{d_k}$ 로 나눠 스케일을 조절한다.

Multi-Head Attention:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

단일 attention 대신, $Q \cdot K \cdot V$ 를 h 개의 서로 다른 학습된 projection으로 각각 d_k, d_k, d_v 차원으로 변환한 뒤 병렬로 attention을 수행하고 결과를 이어붙인다. 서로 다른 관점에서 동시에 정보를 취합할 수 있게 해주는 설계다. 이 논문은 $h=8$, $d_k=d_v=d_{\text{model}}/h=64$ 를 사용하며, 차원을 나눠 쓰기 때문에 전체 계산 비용은 단일 head와 비슷한 수준으로 유지된다.

Attention이 쓰이는 세 곳

위치	Query 출처	Key/Value 출처	역할
Encoder-decoder attention	디코더 이전 레이어	인코더 출력	디코더의 모든 위치가 입력 시퀀스 전체를 참고
인코더 self-attention	인코더 이전 레이어	인코더 이전 레이어 (동일)	원문 안에서 단어들 사이의 관계 파악
디코더 self-attention	디코더 이전 레이어	디코더 이전 레이어 (동일, 마스킹)	지금까지 생성한 출력 안에서의 관계 파악, 미래 위치는 마스킹으로 차단

4-4. Position-wise Feed-Forward Networks

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad \dots \text{식 (2)}$$

각 위치에 독립적으로, 그리고 동일하게 적용되는 2단 선형 변환(중간에 ReLU)이다. 입력을 $d_{\text{model}}=512$ 에서 $d_{\text{ff}}=2048$ 로 넓혔다가 다시 512로 되돌리는 구조로, 커널 크기 1인 컨볼루션 두 개로도 볼 수 있다. attention이 "단어들 사이 정보를 섞는" 역할이라면, 이 부분은 "섞인 정보를 각 단어별로 독립적으로 한 번 더 가공하는" 역할을 한다. 같은 레이어 안에서 위치와 무관하게 동일한 가중치를 쓰지만, 레이어마다 서로 다른 가중치를 학습한다.

4-5. Embeddings and Softmax

입력·출력 토큰을 d_{model} 차원 벡터로 바꾸는 데 학습된 임베딩을 사용하고, 디코더 출력을 다음 토큰 확률로 바꾸는 데도 학습된 선형 변환 + softmax를 사용한다. 두 임베딩 레이어와 softmax 직전 선형 변환은 동일한 가중치 행렬을 공유하며, 임베딩 레이어에서는 이 가중치에 $\sqrt{d_{\text{model}}}$ 을 곱한다.

4-6. Positional Encoding

Transformer는 recurrence도 convolution도 쓰지 않기 때문에, 시퀀스 안의 순서 정보를 스스로 알 방법이 없다. 이를 보완하기 위해 인코더·디코더 입력 임베딩에 위치 정보를 담은 벡터를 더해준다.

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

d_{model} 의 각 차원마다 서로 다른 속도(주파수)로 진동하는 사인 또는 코사인 곡선을 배정해서, 위치(pos)마다 고유한 숫자 패턴을 만들어낸다. i 가 작을수록(초반 차원) 빠르게 진동하고, i 가 클수록 느리게 진동한다 — 여러 속도의 "눈금자"를 겹쳐 위치를 정밀하게 구분하는 방식이다. 사인/코사인 함수를 택한 이유는, 임의의 고정 거리 k 에 대해 $\text{PE}(\text{pos}+k)$ 가 $\text{PE}(\text{pos})$ 의 선형 함수로 표현될 수 있어 모델이 상대적 위치를 학습하기 쉽고, 학습 때 보지 못한 더 긴 시퀀스 길이에도 대응할 수 있기 때문이다. 학습된 위치 임베딩과 비교했을 때 성능은 거의 동일했다고 밝힌다.

5. Why Self-Attention — 세 가지 비교 기준

self-attention을 recurrent, convolutional 레이어와 비교하기 위해 세 가지 기준을 사용한다: (1) 레이어당 전체 계산 복잡도, (2) 병렬화 가능한 계산량(최소 순차 연산 횟수로 측정), (3) 네트워크 내 장거리 의존성 사이의 최대 경로 길이(짧을수록 학습이 쉬움).

레이어 종류	레이어당 복잡도	순차 연산	최대 경로 길이
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

(n =시퀀스 길이, d =표현 차원, k =컨볼루션 커널 크기, r =제한된 self-attention의 이웃 범위)

self-attention은 모든 위치를 상수 횟수($O(1)$)의 순차 연산만으로 연결한다(릴레이 경주처럼 순서를 기다리는 RNN과 달리, 모두가 동시에 서로를 훑어보는 방식). 시퀀스 길이 n 이 표현 차원 d 보다 작은 경우에는 self-attention이 recurrent보다 더 빠르다. 매우 긴 시퀀스에서는 self-attention을 출력 위치 주변의 이웃 크기 r 로 제한할 수 있는데, 이 경우 경로 길이가 $O(n/r)$ 로 늘지만 계산량은 줄어든다.

컨볼루션 레이어는 커널 폭 k 가 n 보다 작으면 모든 위치 쌍을 한 번에 연결하지 못해, 레이어를 여러 개 쌓아야 한다($O(n/k)$ 또는 dilated의 경우 $O(\log_k(n))$). Separable convolution은 복잡도를 크게 줄일 수 있는데, $k=n$ 인 극단적인 경우 이 복잡도가 정확히 "self-attention + point-wise feed-forward"를 합친 것과 같아진다 — 이것이 이 논문이 채택한 접근이다.

부가적인 이점으로, self-attention은 attention 분포를 통해 모델의 동작을 어느 정도 들여다볼 수 있게 해준다. 실제로 여러 attention head가 서로 다른 역할을 학습하고, 문장의 구문적·의미적 구조와 관련된 행동을 보인다는 것을 부록의 시각화 예시로 보여준다.

6. Training — 학습 세부사항

항목	설정값
학습 데이터	영어-독일어: WMT 2014, 약 450만 문장 쌍, byte-pair encoding(약 37000 토큰 공유 어휘) / 영어-프랑스어: WMT 2014, 3600만 문장, word-piece(32000 토큰)
배치 구성	문장 쌍을 비슷한 길이끼리 묶어 배치, 배치당 약 25000 source + 25000 target 토큰
하드웨어	NVIDIA P100 GPU 8개
학습 시간	base 모델: 100,000 스텝(약 12시간) / big 모델: 300,000 스텝(3.5일)
Optimizer	Adam($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=10^{-9}$)
학습률 스케줄	$lr_{rate} = d_{model}^{-0.5} \cdot \min(step^{-0.5}, step \cdot warmup_steps^{-1.5})$, warmup_steps=4000. 초반에는 선형 증가, 이후 스텝 수의 제곱근에 반비례해 감소

항목	설정값
Residual Dropout	각 서브레이어 출력(입력에 더해지기 전) 및 임베딩+positional encoding 합산 값에 적용, base 모델 비율 0.1
Label Smoothing	값 0.1 적용. perplexity는 나빠지지만 정확도·BLEU는 개선

7. 실험 결과

7-1. 기계 번역 성과 (Table 2)

모델	EN-DE BLEU	EN-FR BLEU	학습 비용(FLOPs, EN-DE)
GNMT + RL	24.6	39.92	2.3×10^{19}
ConvS2S	25.16	40.46	9.6×10^{18}
ConvS2S Ensemble	26.36	41.29	7.7×10^{19}
Transformer (base)	27.3	38.1	3.3×10^{18}
Transformer (big)	28.4	41.8	2.3×10^{19}

big 모델은 EN-DE에서 기존 최고 성능(양상블 포함)을 2.0 BLEU 이상 앞질러 새로운 최고 성능(28.4)을 기록했고, base 모델조차 기존 모든 모델·양상블을 능가하면서도 학습 비용은 가장 적었다. EN-FR에서는 big 모델이 41.8 BLEU로 기존 최고 성능 모델의 1/4 미만 비용으로 단일 모델 기준 최고 성능을 달성했다.

7-2. 모델 변형(ablation) 실험 (Table 3)

변경 항목	관찰
Attention head 수 (계산량 고정)	head 1개는 최적 대비 -0.9 BLEU, head가 너무 많아도 품질 저하 — 적정 수의 head가 필요
Attention key 차원(d_k) 축소	모델 품질 저하 — 단순 내적보다 더 정교한 호환성 함수가 도움이 될 수 있음을 시사
모델 크기 확대	예상대로 더 큰 모델이 더 좋음
Dropout	과적합 방지에 매우 유용
사인/코사인 → 학습된 positional embedding	거의 동일한 결과

7-3. 영어 구문 분석 전이 실험 (Table 4)

번역 외 과제로의 일반화를 확인하기 위한 실험. 구문 분석은 출력이 강한 구조적 제약을 가지고 입력보다 훨씬 길다는 점에서 도전적이며, RNN 시퀀스-투-시퀀스 모델은 데이터가 적

을 때 최고 성능을 내지 못했었다는 배경이 있다.

d_model=1024, 4층 Transformer를 Penn Treebank WSJ(약 4만 문장)로 학습, 그리고 약 1700만 문장 규모의 준지도학습 설정으로도 학습했다. 특별한 과제별 튜닝 없이도 대부분의 기존 모델을 능가했고(RNN Grammar 모델만 예외), WSJ 4만 문장만으로 학습했을 때도 RNN 시퀀스-투-시퀀스 모델과 달리 Berkeley-Parser보다 좋은 성능을 냈다 — 데이터가 적은 상황에서도 잘 작동한다는 근거이다.

8. Conclusion 요약

Transformer는 인코더-디코더 구조에서 흔히 쓰이던 순환 레이어를 multi-headed self-attention으로 완전히 대체한 최초의 시퀀스 변환 모델이다. 번역 과제에서 RNN·CNN 기반 아키텍처보다 훨씬 빠르게 학습되면서도 EN-DE·EN-FR 모두 새로운 최고 성능을 달성했다. 저자들은 attention 기반 모델의 향후 방향으로, 텍스트 외 다른 입출력 형태(이미지·오디오·비디오)로의 확장, 이를 효율적으로 다루기 위한 지역적으로 제한된 attention 메커니즘 연구, 그리고 생성 과정을 덜 순차적으로 만드는 것을 꼽는다. 학습·평가 코드는 tensor2tensor라는 이름으로 공개했다.

9. 한계

- **시퀀스 길이에 대한 이차(quadratic) 복잡도:** self-attention의 레이어당 복잡도는 $O(n^2 \cdot d)$ 로, 시퀀스 길이가 매우 길어지면(예: 문서 단위, 이미지·오디오 등 긴 시퀀스) 연산량과 메모리 사용량이 급격히 늘어난다. 논문 스스로도 이 문제를 인지해 "제한된 self-attention"이라는 대안을 언급하지만, 이건 구체적으로 실험하지 않고 향후 연구로 남긴 상태다.
- **positional encoding의 정당화가 다소 정성적:** 사인/코사인 함수를 택한 이유(상대적 위치 학습 용이, 긴 시퀀스로의 extrapolation)는 논리적으로는 설득력이 있지만, 학습된 위치 임베딩과 비교해 "거의 동일한 결과"였다고만 보고할 뿐, extrapolation 성능 자체를 직접 검증하는 실험은 제시되지 않는다.
- **검증 범위가 번역과 구문 분석에 한정:** 결론에서 이미지·오디오·비디오 등으로의 확장을 향후 과제로 언급한다는 것 자체가, 이 논문 시점에는 텍스트 기반 과제 밖에서의 일반화가 검증되지 않았다는 뜻이다.

10. 느낀 점

이 논문을 읽으면서 가장 인상 깊었던 건, "RNN 없이 정말 attention만으로 될까?"라는 질문에서 출발해서 그걸 실제로 증명해냈다는 점이였다. Query/Key/Value라는 단순한 개념

몇 가지를 조합해서, 순서대로 계산해야만 했던 RNN의 근본적인 제약을 완전히 걷어냈다는 게 흥미로웠다.

동시에 이 논문이 구조적 설계 하나하나에 다 이유가 있다는 점도 인상적이었다. $\sqrt{d_k}$ 로 나눠주는 스케일링, multi-head로 나눠 여러 관점을 보게 하는 것, 사인·코사인을 곱라 위치 정보를 주입하는 것까지, 사소해 보이는 선택 하나하나가 전부 설계의 근거를 가지고 있다는 걸 알게 되었다. 다만 한계 섹션에서 짚었듯이, 이런 설계 선택들이 특정 벤치마크에서만 검증됐다는 점에서 만능은 아니라는 것도 함께 알 수 있었다.

앞으로는 이 논문처럼 복잡한 문제를 해결할 때, 기존에 당연하게 여겨지던 전제(RNN은 순차적이어야 한다는 것) 자체를 의심해보는 태도와, 새로운 구조를 제안할 때 각 구성 요소의 선택 이유를 명확하게 설명하고 실험으로 뒷받침하는 방식을 배우고 싶다는 생각이 들었다.
